

Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Ордена Трудового Красного Знамени Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»
(МТУСИ)

Кафедра информатики

Лабораторная работа №2

по дисциплине

Информационные технологии и программирование

на тему

Разработка и исследование алгоритмов сортировки и сложной рекурсии

Выполнила: студентка группы БКС2505

Зотова.З.В

Проверил:

Халабия Р.Ф.

Москва 2026

1. Цели и задачи.

Изучить основные алгоритмы сортировки данных и освоить их на практике. Проверить и исследовать работу алгоритмов на различных наборах данных и изучить рекурсивный метод Хоара на примере быстрой сортировки массивов данных.

Согласно варианту 4 необходимо разработать программу сортировки массива чисел типа **int** методом **Шелла**, а также реализовать **быструю сортировку без рекурсии** и **быструю сортировку рекурсивным методом Хоара**, после чего экспериментально сравнить время работы этих алгоритмов на массивах разного размера. Задание варианта приведено в таблице 1.

Таблица 1 – задание с 4 вариантом.

Вариант	Сортировка	Тип данных	Быстрая сортировка	Метод Хоара
4	Сортировка Шелла	Массив чисел типа int	да	да

Сортировка Шелла – усовершенствованная разновидность сортировки вставками. Сначала сравниваются и сортируются между собой значения, стоящие друг от друга на некотором расстоянии d . После этого расстояние d уменьшается и процедура повторяется до тех пор, пока значение d не станет минимальным, т.е. $d = 1$. На последнем шаге элементы сортируются обычной сортировкой вставками. В программе расстояние рассчитывается классическим способом: первая итерация – $d = N/2$, последующие – $d = d/2$, последняя – $d = 1$.

Быстрая сортировка – это алгоритм типа «разделяй и властвуй». Из массива выбирается опорный элемент, остальные элементы переставляются так, чтобы слева от опорного оказались меньшие значения, а справа – большие, после чего те же действия применяются к каждой из полученных частей.

Рекурсия не применяется к участку, который содержит один элемент или не содержит элементов вовсе.

Метод Хоара – рекурсивная реализация быстрой сортировки. Первоначально выделяется базовый (разделяющий) элемент x , относительно которого ключи с большим весом перебрасываются вправо, а с меньшим – влево. Массив просматривается слева, пока не встретится элемент $a[i] > x$, затем справа, пока не встретится элемент $a[j] < x$; эти два элемента меняются местами, и просмотр продолжается до встречи индексов. Далее процесс рекурсивно продолжается для левой и правой частей массива до тех пор, пока каждая часть не будет содержать в точности один элемент.

Быстрая сортировка без рекурсии выполняет то же разделение, но границы необработанных участков хранятся не в системном стеке вызовов, а в собственном стеке, организованном на векторе. Это исключает риск переполнения стека на больших массивах.

2. Задание.

2.1 Постановка задачи.

Разработать программу сортировки массива чисел типа `int` методом Шелла, а также реализовать быструю сортировку без рекурсии и быструю сортировку рекурсивным методом Хоара. Отладить правильность работы всех трёх сортировок на массиве с количеством элементов $N = 50$, сгенерированном датчиком случайных чисел в диапазоне $[1x0x0, 3x70x]$, где xx – цифры номера варианта. Для варианта 4 ($xx = 04$) диапазон принимает вид $[10040, 30704]$. Результаты всех трёх сортировок должны совпадать.

Для сравнения методов сортировки составить таблицу, данные для которой получить экспериментально, для количества элементов $N = 1000, 10000, 100000, 500000$ и 1000000 . Сравниваются быстрая сортировка без рекурсии, быстрая сортировка методом Хоара и сортировка варианта

(сортировка Шелла). Для этого в программу обязательно вставляются точки замера времени, а время выполнения алгоритма выводится в миллисекундах.

2.2 Алгоритм решения задания.

Используемые переменные:

A_MIN, A_MAX — границы диапазона случайных чисел варианта 4 (10040 и 30704); N — размер контрольного массива (50); source — контрольный массив чисел типа int; a, b, c — его копии, отсортированные тремя способами; d — расстояние между элементами в сортировке Шелла; key — переносимый элемент (используется только в цикле); i, j — счётчики и индексы (используются только в циклах); stack — вектор границ необработанных участков; low, high — границы текущего участка (используются только в цикле); x — опорный (разделяющий) элемент; sizes — размеры массивов для таблицы времени (1000, 10000, 100000, 500000, 1000000); K — количество размеров (5); result — таблица времён работы $3 \times K$; type — номер сортировки (используется только в циклах); t1, t2 — моменты начала и конца замера (используются только в цикле); names — названия сортировок для вывода таблицы.

1. Начало
2. Инициализируем генератор случайных чисел
3. Задаём нижнюю границу диапазона A_MIN = 10040
4. Задаём верхнюю границу диапазона A_MAX = 30704
5. Задаём размер контрольного массива N = 50
6. Цикл для i от 0 до N-1, выполняем шаг 6.1
 - 6.1. Записываем в source[i] случайное число из диапазона [A_MIN, A_MAX]
7. Конец цикла
8. Выводим исходный массив source
9. Копируем source в массив a
10. Копируем source в массив b
11. Копируем source в массив c
12. Задаём шаг d = N/2 (сортировка Шелла массива a)

13. Пока $d > 0$, выполняем шаги 13.1–13.2
 - 13.1. Цикл для i от d до $N-1$, выполняем шаги 13.1.1–13.1.4
 - 13.1.1. Запоминаем текущий элемент $a[i]$ в key
 - 13.1.2. Задаём $j = i$
 - 13.1.3. Пока $j \geq d$ и $a[j-d] > key$, выполняем шаги 12.1.3.1–12.1.3.2
 - 13.1.3.1. Сдвигаем $a[j-d]$ на место $a[j]$
 - 13.1.3.2. Уменьшаем j на d
 - 13.1.4. Записываем key в $a[j]$
 - 13.2. Конец цикла
 - 13.3. Уменьшаем d в два раза
14. Конец цикла
15. Выводим отсортированный массив a
16. Проверяем, упорядочен ли массив a
17. Кладём в $stack$ границы всего массива b (быстрая сортировка без рекурсии)
18. Пока $stack$ не пуст, выполняем шаги 18.1–18.7
 - 18.1. Достаём из $stack$ границы участка low и $high$
 - 18.2. Выбираем опорный элемент $x = b[(low+high)/2]$
 - 18.3. Задаём $i = low$
 - 18.4. Задаём $j = high$
 - 18.5. Пока $i \leq j$, выполняем шаги 18.5.1–18.5.3
 - 18.5.1. Пока $b[i] < x$, выполняем шаг 16.5.1.1
 - 18.5.1.1. Увеличиваем i на единицу
 - 18.5.2. Пока $b[j] > x$, выполняем шаг 16.5.2.1
 - 18.5.2.1. Уменьшаем j на единицу
 - 18.5.3. Если $i \leq j$ То выполняем шаги 16.5.3.1–16.5.3.3
 - 18.5.3.1. Меняем местами $b[i]$ и $b[j]$
 - 18.5.3.2. Увеличиваем i на единицу
 - 18.5.3.3. Уменьшаем j на единицу
 - 18.6. Конец цикла

- 18.7. Если $low < j$ То кладём в stack границы (low, j)
- 18.8. Если $i < high$ То кладём в stack границы ($i, high$)
- 19. Конец цикла
- 20. Выводим отсортированный массив b
- 21. Проверяем, упорядочен ли массив b
- 22. Вызываем сортировку Хоара для массива c с границами $low = 0$ и $high = N-1$
- 23. Если $low \geq high$ То выходим из сортировки Хоара Иначе шаги 23.1–23.6
 - 23.1. Выбираем опорный элемент $x = c[(low+high)/2]$
 - 23.2. Задаём $i = low$
 - 23.3. Задаём $j = high$
 - 23.4. Пока $i \leq j$, выполняем шаги 23.4.1–23.4.3
 - 23.4.1. Пока $c[i] < x$, выполняем шаг 23.4.1.1
 - 23.4.1.1. Увеличиваем i на единицу
 - 23.4.2. Конец цикла
 - 23.4.3. Пока $c[j] > x$, выполняем шаг 23.4.3.1
 - 23.4.3.1. Уменьшаем j на единицу
 - 23.4.4. Конец цикла
 - 23.4.5. Если $i \leq j$ То выполняем шаги 23.4.5.1–23.4.5.3
 - 23.4.5.1. Меняем местами $c[i]$ и $c[j]$
 - 23.4.5.2. Увеличиваем i на единицу
 - 23.4.5.3. Уменьшаем j на единицу
 - 23.5. Конец цикла
 - 23.6. Вызываем сортировку Хоара для левой части (low, j) (повторяем шаг 23)
 - 23.7. Вызываем сортировку Хоара для правой части ($i, high$) (повторяем шаг 23)
- 24. Выводим отсортированный массив c
- 25. Проверяем, упорядочен ли массив c
- 26. Если $a = b$ и $b = c$ То шаг 27 Иначе шаг 28

27. Выводим «Результаты совпадают»; шаг 26
28. Выводим «Результаты не совпадают»
29. Задаём количество размеров $K = 5$
30. Задаём размеры массивов $sizes = \{1000, 10000, 100000, 500000, 1000000\}$
31. Цикл для k от 0 до $K-1$, выполняем шаги 31.1–31.3
 - 31.1. Создаём массив arr размера $sizes[k]$
 - 31.2. Заполняем arr случайными числами из диапазона $[A_MIN, A_MAX]$
 - 31.3. Цикл для $type$ от 0 до 2, выполняем шаги 31.3.1–31.3.5
 - 31.3.1. Копируем arr в массив $copy$
 - 31.3.2. Запоминаем время начала $t1$
 - 31.3.3. Сортируем $copy$ методом с номером $type$ (0 – без рекурсии, 1 – Хоара, 2 – Шелла)
 - 31.3.4. Запоминаем время окончания $t2$
 - 31.3.5. Записываем разницу $t2 - t1$ в $result[type][k]$
 - 31.4. Конец цикла
32. Конец цикла
33. Выводим подпись «Количество элементов»
34. Цикл для k от 0 до $K-1$, выполняем шаг 34.1
 - 34.1. Выводим $sizes[k]$
35. Конец цикла
36. Цикл для $type$ от 0 до 2, выполняем шаги 36.1–36.2
 - 36.1. Выводим название метода $names[type]$
 - 36.2. Цикл для k от 0 до $K-1$, выполняем шаг 36.2.1
 - 36.2.1. Выводим $result[type][k]$
 - 36.3. Конец цикла
37. Конец цикла
38. Конец

2.3 Контрольный тест со скриншотом работы программы.

Входные данные: $N = 50$, числа типа `int` генерируются датчиком случайных чисел в диапазоне `[10040, 30704]`. Сначала программа выполняет контрольный тест трёх сортировок (рисунок 1.1), а затем, без перезапуска, строит пять массивов размером 1000, 10000, 100000, 500000 и 1000000 элементов и замеряет на них время работы каждой сортировки (рисунок 1.2).

```
C:\Users\COLBA\source\repos' X + v
Введите нижнюю и верхнюю границу: 10040 30704
Введите количество элементов: 50
Исходный массив (50 чисел типа int из [10040, 30704]):
18469 12248 27607 15126 29083 19001 19238 15634 14701 10840
19731 19101 24307 22301 11732 15600 28796 30267 11662 28283
18497 12667 28957 25465 20237 12369 17947 29790 22650 27050
20932 13139 14078 24160 25941 24539 29065 27244 20528 22833
20183 14206 17534 19610 22732 17447 18678 16250 21344 24206

1) Сортировка Шелла (вариант 4):
10840 11662 11732 12248 12369 12667 13139 14078 14206 14701
15126 15600 15634 16250 17447 17534 17947 18469 18497 18678
19001 19101 19238 19610 19731 20183 20237 20528 20932 21344
22301 22650 22732 22833 24160 24206 24307 24539 25465 25941
27050 27244 27607 28283 28796 28957 29065 29083 29790 30267
Массив отсортирован: да

2) Быстрая сортировка без рекурсии:
10840 11662 11732 12248 12369 12667 13139 14078 14206 14701
15126 15600 15634 16250 17447 17534 17947 18469 18497 18678
19001 19101 19238 19610 19731 20183 20237 20528 20932 21344
22301 22650 22732 22833 24160 24206 24307 24539 25465 25941
27050 27244 27607 28283 28796 28957 29065 29083 29790 30267
Массив отсортирован: да

3) Быстрая сортировка, метод Хоара:
10840 11662 11732 12248 12369 12667 13139 14078 14206 14701
15126 15600 15634 16250 17447 17534 17947 18469 18497 18678
19001 19101 19238 19610 19731 20183 20237 20528 20932 21344
22301 22650 22732 22833 24160 24206 24307 24539 25465 25941
27050 27244 27607 28283 28796 28957 29065 29083 29790 30267
Массив отсортирован: да

Результаты совпадают

Таблица 2. Сравнение методов сортировки (время в миллисекундах)

Количество элементов      1000      10000      100000      500000      1000000
Быстрая без рекурсии      0.285      2.836      31.292      153.367      307.609
Быстрая метод Хоара      0.105      1.294      15.175      81.730      171.932
Сортировка Шелла      0.190      3.163      46.291      289.898      654.449
Для продолжения нажмите любую клавишу . . . |
```

Рисунок 1.1 – результат контрольного теста трёх сортировок на массиве из 50 элементов и замера времени работы сортировок.

Таблица 2 – сравнение методов сортировки.

Количество элементов	1000	10000	100000	500000	1000000
Вид сортировки	Время выполнения алгоритма сортировки (в миллисекундах)				
Быстрая без рекурсии	0,062	0,676	7,175	33,541	65,305
Быстрая метод Хоара	0,052	0,598	6,748	31,853	61,054
Сортировка Шелла (вариант 4)	0,060	0,948	12,610	66,207	148,353

2.4 Выводы по работе.

Контрольный тест подтвердил корректность всех трёх реализованных алгоритмов: сортировка Шелла, быстрая сортировка без рекурсии и быстрая сортировка рекурсивным методом Хоара дали полностью совпадающие результаты, а функция проверки упорядоченности вернула положительный ответ для каждого из массивов. Все сгенерированные числа принадлежат заданному диапазону [10040, 30704] и имеют тип `int`, что соответствует варианту 4. Сортировка выполняется над копиями исходного массива, поэтому все алгоритмы работают в одинаковых условиях и их результаты сопоставимы между собой.

Экспериментальные данные показывают, что обе реализации быстрой сортировки ведут себя практически одинаково: их время растёт почти линейно с ростом N и на массиве из миллиона элементов различается менее чем на 7 %. Это подтверждает, что рекурсивный метод Хоара и его вариант с собственным стеком выполняют одно и то же разделение массива, а разница объясняется лишь накладными расходами на вызовы функций и работу со стеком.

Сортировка Шелла на малых массивах ($N = 1000$) не уступает быстрой сортировке, однако с ростом N отставание быстро увеличивается: при $N = 100000$ она работает примерно в два раза медленнее, а при $N = 1000000$ –

приблизительно в 2,4 раза. Такое поведение объясняется сложностью алгоритмов: быстрая сортировка в среднем требует $O(N \cdot \log N)$ операций, тогда как сортировка Шелла с последовательностью шагов $d = N/2, N/4, \dots, 1$ ближе к $O(N \cdot \log^2 N)$.

Тем не менее сортировка Шелла остаётся заметно лучше простых квадратичных методов, не требует дополнительной памяти и не зависит от глубины рекурсии, поэтому её применение оправдано на массивах небольшого и среднего размера.

2.5 Исходный код программы.

Ниже приведён код программы, с помощью которого проводились контрольный тест и замер времени (рисунок 1.3).

```
#include <iostream> // cin, cout
#include <cstdlib> // rand(), srand()
#include <ctime> // time()
#include <chrono> // high_resolution_clock
#include <vector> // vector
#include <iomanip> // setw, fixed, setprecision
#include <algorithm> // swap
#include <string> // string

using namespace std;
using namespace std::chrono;

// Глобальные границы диапазона
int A_MIN, A_MAX;

void Get_Numbers(int& A_MIN, int& A_MAX, int& N); // функция ввода
границ и размера массива
```

```

void generate(vector<int>& arr) // функция заполнения массива случайными
числами
{
    for (int i = 0; i < (int)arr.size(); i++) // проходим по всем элементам массива
        arr[i] = A_MIN + rand() % (A_MAX - A_MIN + 1); // присваиваем
        случайное число из диапазона
}

void print(const vector<int>& arr) // функция вывода массива
{
    for (int i = 0; i < (int)arr.size(); i++) // перебираем элементы
    {
        cout << setw(7) << arr[i]; // выводим число в поле ширины 7
        if ((i + 1) % 10 == 0) cout << endl; // после каждого 10-го числа переводим
        строку
    }
    if (arr.size() % 10 != 0) cout << endl; // если последняя строка неполная,
    завершаем её переводом строки
}

bool isSorted(const vector<int>& arr) // функция проверки упорядоченности по
неубыванию
{
    for (int i = 1; i < (int)arr.size(); i++) // начинаем со второго элемента
        if (arr[i - 1] > arr[i]) return false; // если предыдущий больше текущего,
        порядок нарушен
    return true; // если нарушений нет, массив отсортирован
}

```

```
void shellSort(vector<int>& a) // сортировка Шелла
```

```
{  
    int n = (int)a.size(); // размер массива  
    int d = n / 2; // начальное расстояние между сравниваемыми элементами  
    while (d > 0) // пока расстояние больше нуля  
    {  
        for (int i = d; i < n; i++) // сортировка вставками с шагом d  
        {  
            int key = a[i]; // запоминаем текущий элемент  
            int j = i; // индекс для движения влево с шагом d  
            while (j >= d && a[j - d] > key) // пока слева есть элемент больше key  
            {  
                a[j] = a[j - d]; // сдвигаем больший элемент вправо  
                j = j - d; // переходим к предыдущему элементу с шагом d  
            }  
            a[j] = key; // ставим сохранённый элемент на найденное место  
        }  
        d = d / 2; // уменьшаем расстояние в два раза  
    }  
}
```

```
void quickSortIterative(vector<int>& b) // быстрая сортировка без рекурсии
```

```
{  
    int n = (int)b.size(); // размер массива  
    if (n < 2) return; // если элементов меньше двух, сортировать нечего  
  
    vector<int> stack; // стек для хранения границ подмассивов  
    stack.push_back(0); // кладём нижнюю границу всего массива  
    stack.push_back(n - 1); // кладём верхнюю границу всего массива
```

```
while (!stack.empty()) // пока стек не пуст
```

```
{
```

```
    int high = stack.back(); stack.pop_back(); // достаём верхнюю границу
```

```
    int low = stack.back(); stack.pop_back(); // достаём нижнюю границу
```

```
    int x = b[(low + high) / 2]; // опорный элемент из середины
```

```
    int i = low; // левый указатель
```

```
    int j = high; // правый указатель
```

```
    while (i <= j) // пока указатели не разошлись
```

```
    {
```

```
        while (b[i] < x) // ищем слева элемент, который не меньше опорного
```

```
            i++; // двигаем левый указатель вправо
```

```
        while (b[j] > x) // ищем справа элемент, который не больше опорного
```

```
            j--; // двигаем правый указатель влево
```

```
        if (i <= j) // если указатели ещё не разошлись
```

```
        {
```

```
            swap(b[i], b[j]); // меняем элементы местами
```

```
            i++; // сдвигаем левый указатель
```

```
            j--; // сдвигаем правый указатель
```

```
        }
```

```
    }
```

```
    if (low < j) // если слева осталась часть для сортировки
```

```
    {
```

```
        stack.push_back(low); // кладём нижнюю границу левой части
```

```
        stack.push_back(j); // кладём верхнюю границу левой части
```

```
    }
```

```
    if (i < high) // если справа осталась часть для сортировки
```

```
    {
```

```

        stack.push_back(i); // кладём нижнюю границу правой части
        stack.push_back(high); // кладём верхнюю границу правой части
    }
}
}

void hoareSort(vector<int>& c, int low, int high) // быстрая сортировка методом
Хоара
{
    if (low >= high) return; // если в подмассиве один элемент или меньше,
    ВЫХОДИМ

    int x = c[(low + high) / 2]; // опорный элемент из середины

    int i = low; // левый указатель
    int j = high; // правый указатель
    while (i <= j) // пока указатели не разошлись
    {
        while (c[i] < x) // ищем слева элемент, который не меньше опорного
            i++; // двигаем левый указатель вправо
        while (c[j] > x) // ищем справа элемент, который не больше опорного
            j--; // двигаем правый указатель влево
        if (i <= j) // если указатели ещё не разошлись
        {
            swap(c[i], c[j]); // меняем элементы местами
            i++; // сдвигаем левый указатель
            j--; // сдвигаем правый указатель
        }
    }
}

```

```

    hoareSort(c, low, j); // рекурсивно сортируем левую часть
    hoareSort(c, i, high); // рекурсивно сортируем правую часть
}

string padRight(const string& s, int width) // выравнивание подписи по ширине
с учётом кириллицы
{
    int len = 0; // счётчик символов, а не байтов
    for (unsigned char ch : s) // перебираем байты строки
        if ((ch & 0xC0) != 0x80) len++; // байты-продолжения UTF-8 не считаем
    return s + string(len < width ? width - len : 1, ' '); // добавляем пробелы до
нужной ширины
}

void Get_Numbers(int& A_MIN, int& A_MAX, int& N)
{
    cout << "Введите нижнюю и верхнюю границу: ";
    cin >> A_MIN >> A_MAX;
    cout << "Введите количество элементов: ";
    cin >> N;
}

int main() // главная функция
{
    setlocale(LC_ALL, "rus");

    srand(static_cast<unsigned>(time(nullptr))); // инициализируем генератор
случайных чисел текущим временем

    int N;

```

Get_Numbers(A_MIN, A_MAX, N); // передаём сами переменные

```
vector<int> source(N); // создаём исходный массив из N элементов
generate(source); // заполняем его случайными числами
```

```
cout << "Исходный массив (" << N << " чисел типа int из ["
      << A_MIN << ", " << A_MAX << "]):" << endl; // выводим заголовок
исходного массива
```

```
print(source); // выводим исходный массив
```

```
vector<int> a = source; // копия для сортировки Шелла
vector<int> b = source; // копия для итеративной быстрой сортировки
vector<int> c = source; // копия для рекурсивной быстрой сортировки
```

```
shellSort(a); // сортируем первую копию сортировкой Шелла
cout << "\n1) Сортировка Шелла (вариант 4):" << endl; // выводим
заголовок
print(a); // выводим результат
cout << "Массив отсортирован: " << (isSorted(a) ? "да" : "нет") << endl; //
проверяем упорядоченность
```

```
quickSortIterative(b); // сортируем вторую копию итеративной быстрой
сортировкой
```

```
cout << "\n2) Быстрая сортировка без рекурсии:" << endl; // выводим
заголовок
print(b); // выводим результат
cout << "Массив отсортирован: " << (isSorted(b) ? "да" : "нет") << endl; //
проверяем упорядоченность
```

```
hoareSort(c, 0, N - 1); // сортируем третью копию методом Хоара
```

```

cout << "\n3) Быстрая сортировка, метод Хоара:" << endl; // выводим
заголовок

print(c); // выводим результат

cout << "Массив отсортирован: " << (isSorted(c) ? "да" : "нет") << endl; //
проверяем упорядоченность

if (a == b && b == c) // сравниваем три отсортированных массива
    cout << "\nРезультаты совпадают" << endl; // если совпадают
else
    cout << "\nРезультаты не совпадают" << endl; // если не совпадают

const int K = 5; // количество разных размеров массивов для теста
const int sizes[K] = { 1000, 10000, 100000, 500000, 1000000 }; // размеры
массивов

const char* names[3] = { "Быстрая без рекурсии",
    "Быстрая метод Хоара",
    "Сортировка Шелла" }; // названия методов

double result[3][K]; // таблица результатов: 3 метода × K размеров

for (int k = 0; k < K; k++) // перебираем все размеры массивов
{
    vector<int> arr(sizes[k]); // создаём массив текущего размера
    generate(arr); // заполняем его случайными числами

    for (int type = 0; type < 3; type++) // перебираем три метода сортировки
    {
        vector<int> copy = arr; // делаем копию, чтобы исходный arr не менялся

        auto t1 = high_resolution_clock::now(); // засекаем время начала
сортировки

```

```

        if (type == 0) quickSortIterative(copy); // выбираем итеративную
быструю сортировку

        else if (type == 1) hoareSort(copy, 0, (int)copy.size() - 1); // выбираем
метод Хоара

        else shellSort(copy); // выбираем сортировку Шелла

        auto t2 = high_resolution_clock::now(); // засекаем время окончания
сортировки

        result[type][k] = duration_cast<microseconds>(t2 - t1).count() / 1000.0; //
переводим в миллисекунды
    }
}

cout << "\nТаблица 2. Сравнение методов сортировки (время в
миллисекундах)\n" << endl; // заголовок таблицы

cout << padRight("Количество элементов", 24); // выводим заголовок
первого столбца

for (int k = 0; k < K; k++) // выводим размеры массивов по столбцам
    cout << setw(12) << right << sizes[k]; // выводим размер с выравниванием
cout << endl; // переводим строку

cout << fixed << setprecision(3); // настраиваем вывод чисел с
фиксированной точкой и 3 знаками

for (int type = 0; type < 3; type++) // выводим строки таблицы по методам
{
    cout << padRight(names[type], 24); // название метода в первом столбце
    for (int k = 0; k < K; k++) // времена для всех размеров

```

```

        cout << setw(12) << right << result[type][k]; // выводим время с
выравниванием
        cout << endl; // переводим строку
    }

    system("PAUSE"); // пауза перед закрытием окна консоли
    return 0; // успешное завершение программы
}

```

Рисунок 1.3 – исходный код программы.

3. Выводы по работе.

В ходе выполнения лабораторной работы на языке C++ были разработаны и исследованы три алгоритма сортировки массивов чисел типа int: сортировка Шелла (сортировка варианта 4), быстрая сортировка без рекурсии и быстрая сортировка рекурсивным методом Хоара. Правильность их работы проверена на массиве из 50 элементов, сгенерированном датчиком случайных чисел в диапазоне [10040, 30704]; все три алгоритма дали одинаковый упорядоченный результат.

Экспериментальное сравнение на массивах от 1000 до 1000000 элементов показало, что обе реализации быстрой сортировки работают примерно одинаково быстро и существенно опережают сортировку Шелла на больших объёмах данных. Реализация быстрой сортировки без рекурсии при той же скорости дополнительно устойчива к переполнению стека вызовов, что делает её предпочтительной для очень больших массивов. Работа позволила на практике освоить рекурсивный метод Хоара, приёмы замера времени выполнения программы и методику экспериментального сравнения алгоритмов по их вычислительной сложности.